517321 PRINCIPLES OF PROGRAMMING LANGUAGES

RUST TUTORIAL PRESENTATION

กลุ่ม 19 - Closures & Functional Programming

1. นายก้องภพ คุณดำรงชัย - Concept + Short Code Illustration
2. นายคีรเทพ ก้องสุวรรณ - Detailed Code + Live Demo
3. นางสาวจิณณพัต แหล่งหล้า - Rust vs Other Language + PPL Analysis
4. นางสาวจุฑารัตน์ รู้วงษ์ - Exercises + Common Mistakes + Challenge

คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร | ภาคการศึกษาที่ 1 / 2569

Introduction



Functional Programming

- แนวคิดการเขียนโปรแกรมที่เน้นการใช้ ฟังก์ชัน ในการประมวลผล
- แบ่งการทำงานเป็นส่วนย่อย ๆ
- ช่วยให้โค้ด นำกลับมาใช้ซ้ำและจัดการได้ง่าย

Closure

- ฟังก์ชันรูปแบบหนึ่งที่สามารถสร้างขึ้นภายในโปรแกรม
- สามารถ เข้าถึงตัวแปรจากบริบทภายนอก
- ใน Rust สามารถใช้ร่วมกับ Functional Programming ได้

PART 1 - MEMBER 1

Key concept + Short code Illustration

Functional Programming

: เป็นแนวคิดการเขียนโปรแกรมที่เน้นการใช้ฟังก์ชันในการประมวลผลข้อมูล และแบ่งการทำงานออกเป็นส่วนย่อย ๆ เพื่อให้โค้ดสามารถนำกลับมาใช้ซ้ำได้

Why

: แบ่งงานเป็นฟังก์ชัน และนำกลับมาใช้ซ้ำ

Problem Solved

: ลดโค้ดซ้ำ ลดความซับซ้อน และดูแลโค้ดง่ายขึ้น

Key Point

: Function เป็นหน่วยหลักในการจัดการการทำงานของโปรแกรม

Example : Functional Programming

```rust
fn add(a: i32, b: i32) -> i32 {
    a + b
}

▶ Run | ⚙ Debug
fn main() {
    let result: i32 = add(a: 5, b: 3);
    println!("{}", result);
}
```



Expected output : 8

PART 1 - MEMBER 1

Key concept + Short code Illustration

Closure

: Closure คือฟังก์ชันแบบไม่ระบุชื่อที่สามารถกำหนดขึ้นภายในโปรแกรม โดยมีรูปแบบการเขียนที่กระชับ และสามารถนำไปเก็บไว้ในตัวแปรเพื่อเรียกใช้งานได้

Why

: ช่วยลดรูปฟังก์ชันให้เขียนง่ายและกระชับ

Problem Solved

: ลดโค้ดซ้ำและไม่ต้องสร้างฟังก์ชันแยกสำหรับงานเล็ก ๆ

Key Point

: Closure คือฟังก์ชันสั้น ๆ ที่สามารถนำตัวแปรจากภายนอกมาใช้ได้

Example : Closure

```rust
fn main() {
    let add: impl Fn(i32, i32) -> i32 = |a: i32, b: i32| a + b;

    let result: i32 = add(a: 10, b: 2);
    println!("{}", result);
}
```



Expected output : 12

PART 1 - MEMBER 1

Key concept + Short code Illustration

Closure Parameters

: Closure Parameters คือพารามิเตอร์ที่ใช้รับค่าเข้า Closure เพื่อนำไปประมวลผล โดยสามารถกำหนดชนิดข้อมูลได้

Why

: ช่วยกำหนดค่าที่ Closure ต้องการรับ

Problem Solved

: ทำให้ Closure รับข้อมูลและนำไปประมวลผลได้อย่างชัดเจน

Key Point

: Parameter = ค่าที่ส่งเข้าไปให้ Closure ใช้งาน

Example : Closure Parameters

```
1  fn main() {
2      let add: impl Fn(i32, i32) -> i32 = |x: i32, y: i32| x + y;
3
4      let result: i32 = add(x: 10, y: 4);
5
6      println!("{}", result);
7  }
```

x และ y = Parameter
i32 = Data type ของ Parameter

x + y = การประมวลผลของ Closure

Expected output : 14

PART 1 - MEMBER 1

Key concept + Short code Illustration

Closure Capture

: Closure ใน Rust สามารถเข้าถึงตัวแปรที่อยู่นอกขอบเขตของ Closure ได้ ความสามารถนี้เรียกว่า Closure Capture

Why

: ทำให้ Closure สามารถใช้ตัวแปรจากภายนอกได้

Problem Solved

: ไม่ต้องส่งตัวแปรภายนอกเข้าไปเป็น Parameter ทุกครั้ง

Key Point

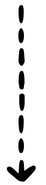
: Closure สามารถจดจำและนำตัวแปรภายนอกมาใช้งานได้

Example : Closure Capture

```rust
fn main() {
    let x: i32 = 10;

    let add_x: impl Fn(i32) -> i32 = |n: i32| n + x;

    println!("{}", add_x(5));
}
```



Expected output : 15

PART 1 - MEMBER 1

Key concept + Short code Illustration

Higher-Order Functions

: Higher-Order Function คือฟังก์ชันที่สามารถ รับฟังก์ชันอื่นเข้ามา หรือ ส่งฟังก์ชันกลับออกไปได้ช่วยให้เราสามารถ ส่งพฤติกรรมการทำงานเข้าไปให้ฟังก์ชันจัดการได้

Why

: ช่วยให้สามารถส่งและจัดการพฤติกรรมของโปรแกรมผ่านฟังก์ชันได้

Problem Solved

: แก้ปัญหาการต้องเขียนฟังก์ชันหลายตัวที่มีโครงสร้างคล้ายกัน โดยสามารถส่งฟังก์ชันเป็นข้อมูลได้

Key Point

: ฟังก์ชันสามารถรับฟังก์ชันเป็น argument หรือคืนค่าฟังก์ชันกลับมาได้

Example : Higher-Order Functions

```rust
fn apply(value: i32, operation: fn(i32) -> i32) -> i32 {
    operation(value)
}

fn double(x: i32) -> i32 {
    x * 2
}

▶ Run | ⚙ Debug
fn main() {
    let result: i32 = apply(value: 10, operation: double);
    println!("{}", result);
}
```

Expected output : 20

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 1

1.1 Fn: ยืมอ่านอย่างเดียว

```
fn demo_closure_traits() {
    println!("\n--- [Demo 1] Closure Traits (Fn, FnMut, FnOnce) -
--");

    // 1.1 Fn: ยืมอ่านอย่างเดียว (Immutable Borrow)
    let greeting = String::from("Hello");
    let print_greeting = || println!("Fn trait: {}", greeting);
    print_greeting();
    println!("ค่า greeting ยังใช้ต่อได้: {}", greeting); // compile
ผ่านเพราะแค่ยืมอ่าน
```

> Fn ยืมอ่านอย่างเดียว

> greeting ยังใช้ต่อได้ เพราะแค่ยืมอ่าน

OUTPUT

```
--- [Demo 1] Closure Traits (Fn, FnMut,
FnOnce) ---
Fn trait: Hello
ค่า greeting ยังใช้ต่อได้: Hello
```

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 1

1.2 FnMut: ยืมแบบแก้ไขค่าได้

```
// 1.2 FnMut: ยืมแบบแก้ไขค่าได้ (Mutable Borrow)
let mut count = 0;
let mut increment = || {
    count += 1;
    println!("FnMut trait count: {}", count);
};
increment();
increment();
println!("Final count: {}", count);
```

FnMut แก้ไขค่าที่จับมาได้

ต้องประกาศ **let mut** increment ด้วย

OUTPUT

```
FnMut trait count: 1
FnMut trait count: 2
Final count: 2
```

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 1

1.3 FnOnce: ย้าย ownership

```
    // 1.3 FnOnce: ย้าย Ownership (Move/Consume) รันได้ครั้งเดียว
    let data = vec![1, 2, 3];
    let consume_data = || {
        println!("FnOnce trait: vector length =
{}", data.len());
        drop(data); // data ถูกทำลายทิ้งตรงนี้
    };
    consume_data();
    // consume_data(); //
}
```

FnOnce ย้าย ownership ออกไป (drop)

เรียก consume_data() ซ้ำ → compile error

OUTPUT

```
FnOnce trait: vector length = 3
```

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 2

Concurrency with move

```rust
use std::thread;

fn demo_move_concurrency() {
    println!("\n--- [Demo 2] Concurrency with `move` ---");

    let thread_data = vec![10, 20, 30];

    // หากไม่ใส่ keyword `move` ตัว Compiler จะเตือนว่า data
    // อาจมีอายุสั้นกว่า Thread (Lifetime issue)
    let handle = thread::spawn(move || {
        println!("Thread worker ได้รับ data: {:?}", thread_data);
    });

    handle.join().unwrap();
    // println!("{:?}", thread_data); // Compile Error: ownership
    // ย้ายไปที่ Thread แล้ว
}
```

move ย้าย ownership เข้า thread

ใช้ thread_data ข้างนอกต่อ → compile error

OUTPUT

```
--- [Demo 2] Concurrency with `move` ---
Thread worker ได้รับ data: [10, 20, 30]
```

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 3

Returning Closures

```rust
// 3.1 คืนค่า Closure เดี่ยวๆ แบบ Static Dispatch (Zero-cost)
fn create_adder(x: i32) -> impl Fn(i32) -> i32 {
    move |y| x + y
}

// 3.2 คืนค่า Closure ที่หลากหลายผ่าน Dynamic Dispatch (Heap Allocated)
fn make_operation(op: &str) -> Box<dyn Fn(i32, i32) -> i32> {
    if op == "add" {
        Box::new(|a, b| a + b)
    } else {
        Box::new(|a, b| a * b)
    }
}
```

impl Fn คืนแบบเดียว เร็ว ไม่ใช้ heap

Box<dyn Fn> คืนได้หลายแบบ ใช้ heap

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 3

Returning Closures: เรียกใช้

```
fn demo_returning_closures() {
    println!("\n--- [Demo 3] Returning Closures ---");

    let add_five = create_adder(5);
    println!("create_adder(5)(10) = {}", add_five(10));

    let calc = make_operation("multiply");
    println!("make_operation('multiply')(4, 5) = {}",
calc(4, 5));
}
```

create_adder(5) → บวก 5 ทุกค่า

make_operation("multiply") → คูณ

OUTPUT

```
--- [Demo 3] Returning Closures ---
create_adder(5)(10) = 15
make_operation('multiply')(4, 5) = 20
```

PART 2 - MEMBER 2

Detail Code + Live Demo

DEMO 4

Functional Pipeline

```
fn demo_functional_pipeline() {
    println!("\n--- [Demo 4] Functional Programming (Zero-Cost
Abstractions) ---");

    let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    // Pipeline: Filter -> Map -> Fold (Declarative Style)
    // Rust จะ compile เป็น Loop ภาษาเครื่องตัวเดียว ไม่มี Heap Overhead
    let sum_of_even_squares: i32 = numbers
        .iter()
        .filter(|&&x| x % 2 == 0) // คัดเฉพาะเลขคู่
        .map(|&x| x * x) // ยกกำลังสอง
        .fold(0, |acc, x| acc + x); // รวมผลลัพธ์

    println!("ผลรวมของเลขคู่ยกกำลังสอง (2^2 + 4^2 + 6^2 + 8^2 + 10^2) =
{}", sum_of_even_squares);
}
```

filter คัดเฉพาะเลขคู่

map ยกกำลังสอง

fold รวมผลทั้งหมด

OUTPUT

```
--- [Demo 4] Functional Programming (Zero-
Cost Abstractions) ---
ผลรวมของเลขคู่ยกกำลังสอง (2^2 + 4^2 + 6^2 +
8^2 + 10^2) = 220
```

PART 3 - MEMBER 3

PPL Perspective
Syntax & Semantics

9.1 Syntax

Closure ใน Rust ใช้เครื่องหมาย | | สำหรับกำหนด parameter และตามด้วย expression หรือ body ของ Closure

```
let add = |x| x + 10;
```

|x|
parameter

x + 10
expression

- Rust สามารถอนุมานชนิดข้อมูลของ parameter และ return value ของ Closure ได้จากบริบท

9.2 Semantics

Closure สามารถ capture ตัวแปรจาก environment ที่มันถูกสร้างขึ้นมาได้

```
let n = 10;
let add_n = |x| x + n ;
```

closure
|x| x + n;

capture →

environment
n = 10;

- Rust จะกำหนดวิธีการ capture ตามการใช้งาน เช่น immutable borrow, mutable borrow หรือ move
- เกี่ยวข้องกับ Scope, Environment และ Binding

PART 3 - MEMBER 3

PPL Perspective
Type System & Memory Management

9.3 Type System

Closure แต่ละตัวมี anonymous type ที่เป็นเอกลักษณ์ของตัวเอง และ Rust ใช้ Closure Traits เพื่อกำหนดลักษณะการเรียกใช้งาน

Trait	ลักษณะ
Fn	เรียกใช้งานซ้ำได้โดยไม่แก้ไขค่าที่ capture
FnMut	เรียกใช้งานซ้ำได้ และสามารถแก้ไขค่าที่ capture ได้
FnOnce	Closure ที่มีการMove ค่าที่captureไว้ ทำให้เรียกใช้งานซ้ำไม่ได้

- Closure จะ implement trait เหล่านี้ตามลักษณะการใช้งานค่าที่ capture ภายใน body

9.4 Memory/Resource

Closures ใน Rust ทำงานร่วมกับระบบ Ownership และ Borrowing ของภาษา

Capture Variable
จาก environment

↓

Ownership/Borrowing
เลือกวิธี capture ที่เหมาะสม

↓

Memory Safety
ตรวจสอบตั้งแต่ compile time

การ capture มีหลายรูปแบบ

- Immutable borrow (&T)
- Mutable borrow (&mut T)
- Move (ย้าย ownership)

↓

ใช้ move เพื่อบังคับให้ Closure capture ค่าแบบ value

PART 3 - MEMBER 3

PPL Perspective
Abstraction & Why Rust

9.5 Abstraction

Closure เป็นตัวอย่างของ **Higher-Order Function** เนื่องจากสามารถส่ง Closure เป็น argument ให้กับ function หรือ method อื่นได้

```rust
let numbers = vec![1, 2, 3];
let doubled: Vec<_> = numbers.iter()
            .map(|x| x * 2)
            .collect();
```

map()

รับ closure เป็น argument

Functional Programming

- เกี่ยวข้องกับ **Functional Programming** เพราะสามารถนำ function หรือ Closure มาใช้เป็นข้อมูลและนำไปประกอบกับการประมวลผลข้อมูล เช่น map() และ filter()

9.6 Why Rust

Rust ออกแบบ Closures ให้ทำงานร่วมกับระบบ Type System, Ownership และ Borrowing ของภาษา

Type System
ตรวจสอบชนิดของข้อมูล

↓

Ownership & Borrowing
จัดการค่าที่ capture

↓

Compile-time Checking
ตรวจสอบการใช้งานตัวแปรและmemory

↓

Memory Safety
ช่วยป้องกัน memory-related errors

- Closures และ Iterators ช่วยให้เขียนโค้ดแบบ Functional Programming ได้กระชับ
- Rust มุ่งเน้นให้ abstraction เหล่านี้มีประสิทธิภาพ โดยไม่เพิ่ม runtime overhead ที่ไม่จำเป็น

PART 3 - MEMBER 3

Rust vs Other Languages

Aspect	Rust	Python	Java	C++
Syntax	ใช้เครื่องหมาย | | สำหรับกำหนด parameter ของ Closure เช่น |x| x + 10	ใช้คำสั่ง lambda สำหรับสร้าง Lambda Expressions เช่น lambda x: x + 10	ใช้ Lambda Expression ในรูปแบบ parameter→ expression เช่น x → x + 10	ใช้ Lambda Expression และกำหนดการ capture ด้วย [ ] เช่น [n](int x) {return x + n; }
Semantics	capture ตัวแปรจาก environmentและ Rust จะกำหนดวิธี capture ตามการใช้งาน	Lambda สามารถเข้าถึงตัวแปรจาก scope ภายนอกได้	Lambda สามารถ capture ตัวแปร local ที่เป็น final หรือ effectively final ได้	Lambda สามารถ capture ตัวแปรภายนอกได้ โดยกำหนดวิธี capture ได้
Type System	เป็น Static Typing และ Closure แต่ละตัวมี type เฉพาะของตัวเอง โดยเกี่ยวข้องกับ Fn, FnMut, FnOnce	เป็น Dynamic Typing โดยชนิดข้อมูลจะถูกตรวจสอบระหว่าง Runtime	เป็น Static Typing และ Lambda ใช้งานผ่าน Functional Interface	เป็น Static Typing และ Lambda แต่ละตัวมี unnamed closure type

PART 3 - MEMBER 3

Rust vs Other Languages

Aspect	Rust	Python	Java	C++
Memory Management	ใช้ Ownership, Borrowing และ move ในการจัดการค่าที่ capture	ใช้ Garbage Collection เพื่อจัดการ memory อัตโนมัติ	ใช้ Garbage Collection เพื่อจัดการ memory อัตโนมัติ	ใช้ RAII และสามารถควบคุมการจัดการ resource ได้ละเอียด
Safety	Compiler ตรวจสอบ Type, Ownership และ Borrowing ตั้งแต่ Compile time ช่วยป้องกัน memory-related errors	มีการตรวจสอบระหว่าง Runtime และ Garbage Collection แต่ไม่มีระบบ Ownership แบบ Rust	มี Static Type Checking และ Garbage Collection	มี Static Type Checking และ RAII แต่ผู้เขียนยังต้องระวัง lifetime และpointer

Analysis: แต่ละภาษารองรับ Closure/Lambda คล้ายกัน แต่มีแนวทางการออกแบบต่างกัน โดย Rust เน้น Type Safety และ Memory Safety, Python เน้นความยืดหยุ่น, ส่วน Java และ C++ ใช้ Static Typing แต่มีแนวทางจัดการ Lambda และ Memory แตกต่างกัน ตามแนวคิดการออกแบบของแต่ละภาษา

PART 4 - MEMBER 4

Common Mistakes - ข้อผิดพลาดที่พบบ่อย

Example 1 : พยายามคืน Fn(...) ตรง ๆ จากฟังก์ชัน

Incorrect

```
fn factory() -> Fn(i32) -> i32 {
    let num = 5;

    |x| x + num
}

fn main() {
    let f = factory();
    println!("{}", f(1));
}
```

Output

```
error[E0277]: the trait bound Fn(i32) -> i32: Sized is not satisfied
```

Correct

```
fn factory() -> Box<dyn Fn(i32) -> i32> {
    let num = 5;

    Box::new(move |x| x + num)
}

fn main() {
    let f = factory();
    println!("{}", f(1));
}
```

Output

```
6
```

PART 4 - MEMBER 4

Common Mistakes - ข้อผิดพลาดที่พบบ่อย

Example 2 : ลืมใช้ move ตอนคืน Closure ที่ capture ตัวแปรภายในฟังก์ชัน

Incorrect

```
fn factory() -> Box<dyn Fn(i32) -> i32> {
    let num = 5;

    Box::new(|x| x + num)
}

fn main() {
    let f = factory();
    println!("{}", f(1));
}
```

Output

```
error[E0373]: closure may outlive the current function, but it borrows `num`
```

Correct

```
fn factory() -> Box<dyn Fn(i32) -> i32> {
    let num = 5;

    Box::new(move |x| x + num)
}

fn main() {
    let f = factory();
    println!("{}", f(1));
}
```

Output

```
6
```

PART 4 - MEMBER 4

Challenge - คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

วิเคราะห์

```
fn factory(x: i32) -> impl Fn(i32) -> Box<dyn Fn(i32) -> i32> {
    move |y| {
        let z = x + y;
        Box::new(
            move |n| n * z)
    }}
```

คำถาม ?

1.) Closure ชั้นนอก capture อะไร ?
2.) Closure ชั้นใน capture อะไร ?
3.) มี Closure ทั้งหมดกี่ตัว ?
4.) เพราะเหตุใด z จึงต้องใช้ move กับ Closure ชั้นใน ?

คำใบ้ (hint) โจทย์นี้เป็น Higher-Order Function ที่คืน Closure ซ้อนกัน

1.) , 2.) Closure นี้เอาตัวแปรอะไรจากข้างนอกมาใช้, ห้ามนำ parameter ของ Closure มานับเป็น capture

3.) การดูจำนวน Closure ให้นับเครื่องหมาย | |

4.) Closure ชั้นในต้องเอาตัวแปรของชั้นนอกติดตัวออกไปด้วย → นึกถึง move

PART 4 - MEMBER 4

Solution 1 - 2 (Concept เดียวกัน)

1.) Closure ชั้นนอก capture อะไร ?

ตอบ : x

y ไม่ใช่ตัวที่ capture เพราะ y เป็นค่าที่ Closure รับเข้ามาเอง เรียกว่า Parameter แต่ x มาจากข้างนอก Closure จึงเป็นตัวที่ capture

```
fn factory(x)
        ↓
    move |y| {
        x + y
    }
```

|y| เป็น Parameter

```
factory
 │
 └── Closure ชั้นนอก: |y| { ... }
      │
      └── Closure ชั้นใน: |n| n * z
```

2.) Closure ชั้นใน capture อะไร ?

ตอบ : z

ตัวไหนอยู่ใน |n| ? → n เป็น parameter
ตัวไหนไม่ได้อยู่ใน |n| แต่ถูกใช้ ? → z

```
move |n| n * z
```

|n| เป็น Parameter

PART 4 - MEMBER 4

Solution 3 - 4

3.) มี Closure ทั้งหมดกี่ตัว ?

ตอบ : 2 ตัว

มี |...| 2 ชุด

```
move |y| {
  let z = x + y;
  Box::new(
    move |n| n * z)
}
```

ดังนั้นมีทั้งหมด 2 ตัว

1.) move |y| {...} = Closure ชั้นนอก

2.) move |n| n * z = Closure ชั้นใน

4.) เพราะเหตุใด z จึงต้องใช้ move กับ Closure ชั้นใน ?

ตอบ : Closure ชั้นในต้องเก็บค่า z ไว้ใช้หลังจาก Closure ชั้นนอกทำงานเสร็จ

ถ้าไม่ใส่ move

```
Box::new(|n| n * z)
```

เพราะ z ต้องถูกเก็บไว้ใช้ใน Closure ชั้นในหลัง Closure ชั้นนอกจบ จึงต้องใช้ move เพื่อให้ Closure ชั้นในเป็นเจ้าของ z

ใส่ move

```
Box::new(move |n| n * z)
```

เพราะ move ของ Closure ชั้นในจำเป็นเพื่อให้ตัวเองเป็นเจ้าของ z และเก็บ z ไว้ใช้ภายหลัง หลังจาก Closure ชั้นนอกจบการทำงาน

Key Takeaways - สรุปสิ่งที่ควรจำ

- **Capture** — Closure สามารถจับค่า (capture) จาก scope ภายนอกมาใช้ได้
- **Parameter** — รับค่าเข้า Closure ผ่าน parameter เช่น |x| x + 1
- **Type System** — Rust กำหนดชนิดของ Closure และใช้ impl Fn / Box<dyn Fn> เมื่อคืน Closure
- **Ownership** — Closure อาจเป็นเจ้าของค่าที่ capture โดยใช้ move
- **Borrowing** — Closure สามารถยืมค่าจากภายนอกแทนการเป็นเจ้าของได้
- **Higher-Order Function** — ฟังก์ชันสามารถรับหรือคืน Closure ได้
- **Nested Closure** — สามารถสร้าง Closure ซ้อนกัน และส่ง Closure ออกไปใช้งานภายหลังได้
- **move** ใช้เมื่อ Closure ต้อง เป็นเจ้าของค่าที่ capture
- **impl Fn(...)** ใช้สำหรับ คืน Closure จากฟังก์ชัน
- **Box<dyn Fn(...)>** ใช้เมื่อ Closure มีขนาดไม่แน่นอนและต้องการเก็บเป็น trait object
- Closure สามารถ ซ้อนกัน **(Nested Closures)** และส่งต่อเป็นค่าได้
- **Functional Programming** เน้นการใช้ ฟังก์ชันเป็นข้อมูล เพื่อสร้างโค้ดที่ยืดหยุ่นและนำกลับมาใช้ซ้ำได้

REFERENCES - เอกสารอ้างอิง

- **Closures** (จากหนังสือ The Rust Programming Language) — https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/first-edition/closures.html#closures
- **Functional Language Features: Iterators and Closures** — https://rust-book.cs.brown.edu/ch13-00-functional-features.html
- **Closures: Anonymous Functions that Capture Their Environment**— https://rust-book.cs.brown.edu/ch13-01-closures.html
- **Functions** (จากคู่มือ Rust by Example) — https://doc.rust-lang.org/rust-by-example/fn.html
- **Closures** (จากคู่มือ Rust by Example — https://doc.rust-lang.org/rust-by-example/fn/closures.html

Questions & Answers

กลุ่ม 19 - Closures & Functional Programming